SQL Cheatsheet for Zomato Product Analyst Test

Oct 26, 2026 · @Kartikey

1. How to approach the test

Every question reduces to five decisions: the grain of the answer, the filter, the join, the aggregate and the ordering. Settle the grain first (one row per user? per order? per restaurant per day?) and the rest follows.

Logical execution order (why you cannot use a SELECT alias in WHERE):

1. `FROM` / `JOIN`
2. `WHERE` (filters rows, before grouping)
3. `GROUP BY`
4. `HAVING` (filters groups, after aggregation)
5. `SELECT` (window functions run here)
6. `DISTINCT`
7. `ORDER BY` (aliases allowed)
8. `LIMIT` / `OFFSET`

5-step method for any question

1. Restate the metric in one line, with its numerator and denominator.
2. Name the grain of the final output (one row per what?).
3. Pick the tables and the join keys. Check whether any join can duplicate rows.
4. Build in small CTEs, running each one to check it.
5. Sanity-check: do row counts, totals and NULLs look sensible?

Habits that save marks and time

- Ask or state assumptions: how is a cancelled order treated, which time zone, what counts as an active user.
- Always check `COUNT(*)` vs `COUNT(DISTINCT id)` to catch duplicates.
- Write the final `SELECT` columns in the order the question asks for them, with clear aliases.

- Prefer CTEs over deeply nested subqueries; they are easier to debug and to explain.
- When stuck, solve the simpler version first (one user, one day), then generalise.

2. Core syntax

The skeleton of almost every query:

```
SELECT   col1, col2, expr AS alias
FROM     table t
WHERE    condition
GROUP BY col1, col2
HAVING   agg_condition
ORDER BY col1 DESC, col2 ASC
LIMIT    10;
```

Filtering operators

```
WHERE city = 'Delhi' AND status <> 'cancelled'        -- <> or !=
  AND order_value BETWEEN 200 AND 500                  -- inclusive on both
ends
  AND cuisine IN ('North Indian', 'Chinese')
  AND restaurant_name LIKE 'Pizza%'                    -- % = any chars, _ =
one char
  AND rating IS NOT NULL                               -- never = NULL
  AND NOT (delivery_fee = 0 OR is_gold = 1)
```

NULL rules

- Any comparison with NULL is unknown, so `x = NULL` never matches. Use `IS NULL` / `IS NOT NULL`.
- `COALESCE(a, b, 0)` returns the first non-NULL value. `NULLIF(a, 0)` returns NULL when a = 0 (useful to avoid divide-by-zero).
- Aggregates ignore NULLs, except `COUNT(*)`.
- `NOT IN (subquery)` returns nothing if the subquery contains a NULL. Prefer `NOT EXISTS` or a LEFT JOIN anti-join.

CASE WHEN (the most useful tool for bucketing and conditional logic)

```
SELECT order_id,
       CASE WHEN order_value < 200 THEN 'low'
            WHEN order_value < 500 THEN 'mid'
            ELSE 'high' END AS value_bucket
FROM orders;
```

Other basics

Need	Syntax
Unique values	`SELECT DISTINCT city FROM orders`
Count unique	`COUNT(DISTINCT user_id)`
Top N	`ORDER BY x DESC LIMIT N` (SQL Server: `SELECT TOP N`)
Pagination	`LIMIT 10 OFFSET 20`
Combine results	`UNION` (removes duplicates), `UNION ALL` (keeps all, faster)
Common to both queries	`INTERSECT`; in A not B: `EXCEPT` (MySQL lacks both before 8.0.31)
Cast type	`CAST(x AS DECIMAL(10,2))` or `x::numeric` (Postgres)
Round	`ROUND(x, 2)`
Integer division trap	`1 / 2 = 0` in many engines; use `1.0 * a / b`

3. Aggregations

A `GROUP BY` collapses rows into one row per group. Every non-aggregated column in SELECT must appear in GROUP BY.

Function	What it does
`COUNT(*)`	All rows, including NULLs
`COUNT(col)`	Rows where col is not NULL

Function	What it does
`COUNT(DISTINCT col)`	Unique non-NULL values
`SUM`, `AVG`, `MIN`, `MAX`	Ignore NULLs

```
-- Orders, GMV and AOV by city for delivered orders
SELECT city,
       COUNT(*)                        AS orders,
       COUNT(DISTINCT user_id)         AS buyers,
       SUM(order_value)                AS gmv,
       ROUND(AVG(order_value), 2)      AS aov
FROM   orders
WHERE  status = 'delivered'
GROUP  BY city
HAVING COUNT(*) >= 100
ORDER  BY gmv DESC;
```

WHERE vs HAVING. WHERE filters rows before grouping and cannot use aggregates. HAVING filters groups after aggregation. Put every condition you can in WHERE, because it is faster.

Conditional aggregation (pivot-style metrics in one pass, very common in tests)

```
SELECT order_date,
       COUNT(*)                                          AS total_orders,
       SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END) AS cancelled,
       ROUND(100.0 * SUM(status = 'cancelled') / COUNT(*), 2) AS cancel_pct,
-- MySQL shortcut
       AVG(CASE WHEN is_gold = 1 THEN order_value END)  AS gold_aov
FROM   orders
GROUP  BY order_date;
```

In PostgreSQL use `COUNT(*) FILTER (WHERE status = 'cancelled')` instead of the MySQL boolean-sum shortcut.

Rates and ratios. Multiply by `1.0` or `100.0` before dividing to avoid integer division, and guard the denominator with `NULLIF(denominator, 0)`.

GROUP BY extras: `GROUP BY 1, 2` groups by column position; `ROLLUP` adds subtotals and grand totals (MySQL: `WITH ROLLUP`).

4. Joins

Join	Returns	Use when
`INNER JOIN`	Only rows that match in both tables	You need matched data only
`LEFT JOIN`	All left rows, NULLs where the right has no match	Keep every user, even those with no orders
`RIGHT JOIN`	Mirror of LEFT	Rarely needed; swap table order and use LEFT
`FULL OUTER JOIN`	All rows from both sides	Reconciling two sources (not in MySQL)
`CROSS JOIN`	Every combination	Building a date x city grid

```
SELECT u.user_id, u.signup_date, COUNT(o.order_id) AS orders
FROM   users u
LEFT   JOIN orders o
       ON o.user_id = u.user_id
      AND o.status = 'delivered'      -- filter on the right table goes in
ON, not WHERE
GROUP  BY u.user_id, u.signup_date;
```

Anti-join (rows in A with no match in B, e.g. users who never ordered)

```
SELECT u.user_id
FROM   users u
LEFT   JOIN orders o ON o.user_id = u.user_id
WHERE  o.order_id IS NULL;
-- equivalent: WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id =
u.user_id)
```

Self-join (compare rows of the same table, e.g. a user's consecutive orders or pairs of restaurants)

```
SELECT a.order_id, b.order_id AS next_order
FROM   orders a
JOIN   orders b ON b.user_id = a.user_id AND b.order_date > a.order_date;
```

Join traps

- **Fan-out:** joining orders to order_items repeats each order's `order_value` once per item, so `SUM(order_value)` is inflated. Aggregate one side first in a CTE, then join.
- **LEFT JOIN turned INNER:** a `WHERE o.status = 'delivered'` on the right table drops the NULL rows. Move it to `ON`.
- **Non-unique keys:** check `COUNT(*)` vs `COUNT(DISTINCT key)` before joining.
- **`USING (col)`** is shorthand when both tables share the column name.

5. Subqueries and CTEs

A CTE (`WITH`) names an intermediate result so the final query reads top to bottom. Use one whenever a query needs two stages, such as aggregate then filter, or rank then pick.

```
WITH user_orders AS (
    SELECT user_id, COUNT(*) AS n_orders, SUM(order_value) AS spend
    FROM   orders
    WHERE  status = 'delivered'
    GROUP  BY user_id
),
avg_stats AS (
    SELECT AVG(spend) AS avg_spend FROM user_orders
)
SELECT u.user_id, u.n_orders, u.spend
FROM   user_orders u
CROSS  JOIN avg_stats a
WHERE  u.spend > a.avg_spend;
```

Subquery placements

Where	Example
In `WHERE` (scalar)	`WHERE order_value > (SELECT AVG(order_value) FROM orders)`
In `WHERE` (list)	`WHERE user_id IN (SELECT user_id FROM orders WHERE city = 'Mumbai')`
`EXISTS`	`WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.user_id)`
In `FROM`	`FROM (SELECT ... GROUP BY ...) AS t` (derived tables need an alias)

Where	Example
In `SELECT`	Correlated scalar subquery; works but is slow on big tables

Second highest value (classic question)

```
SELECT MAX(order_value) FROM orders
WHERE  order_value < (SELECT MAX(order_value) FROM orders);
-- or, handling ties properly:
SELECT DISTINCT order_value FROM (
    SELECT order_value, DENSE_RANK() OVER (ORDER BY order_value DESC) AS rk
FROM orders
) t WHERE rk = 2;
```

`WITH RECURSIVE` handles hierarchies and generated sequences (for example a calendar of dates). It is less common in analyst tests, so learn it last.

6. Window functions

A window function computes over a set of related rows without collapsing them, so you keep one output row per input row. The form is `func() OVER (PARTITION BY ... ORDER BY ... frame)`. They cannot appear in WHERE, so wrap the query in a CTE and filter outside.

Function	Use
`ROW_NUMBER()`	Unique 1, 2, 3 per partition; ties broken arbitrarily. Best for "latest row per user" and de-duplication
`RANK()`	Ties share a rank, then a gap (1, 1, 3)
`DENSE_RANK()`	Ties share a rank, no gap (1, 1, 2). Best for "Nth highest"
`LAG(col, n)` / `LEAD(col, n)`	Value from n rows before / after. Gaps between orders, month-over-month change
`FIRST_VALUE` / `LAST_VALUE`	First or last value in the frame (LAST_VALUE needs an explicit frame)
`SUM() OVER`, `AVG() OVER`	Running totals, moving averages, share of total

Function	Use
`NTILE(n)`	Splits rows into n equal buckets (quartiles, deciles)

```
-- Top 3 restaurants by revenue in each city
WITH rev AS (
    SELECT city, restaurant_id, SUM(order_value) AS revenue
    FROM   orders GROUP BY city, restaurant_id
)
SELECT * FROM (
    SELECT city, restaurant_id, revenue,
           DENSE_RANK() OVER (PARTITION BY city ORDER BY revenue DESC) AS rk
    FROM rev
) t
WHERE rk <= 3;
```

```
-- Days since the user's previous order
SELECT user_id, order_date,
       order_date - LAG(order_date) OVER (PARTITION BY user_id ORDER BY
order_date) AS days_gap   -- Postgres
       -- MySQL: DATEDIFF(order_date, LAG(order_date) OVER (...))
FROM orders;
```

```
-- Running total, 7-day moving average, and share of city total
SELECT order_date, city, daily_orders,
       SUM(daily_orders) OVER (PARTITION BY city ORDER BY order_date) AS
running_total,
       AVG(daily_orders) OVER (PARTITION BY city ORDER BY order_date
                               ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS
ma_7d,
       100.0 * daily_orders / SUM(daily_orders) OVER (PARTITION BY city) AS
pct_of_city
FROM daily_city_orders;
```

Frame clause. With `ORDER BY` and no frame, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which is why a running total works. Use `ROWS BETWEEN n PRECEDING AND CURRENT ROW` for a fixed-size moving window.

First order of every user: `ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY order_ts) = 1` in an outer query.

7. Date and string functions

The test engine is usually MySQL or PostgreSQL, so know both spellings. If the question does not say, state which one you are assuming.

Task	MySQL	PostgreSQL
Today / now	`CURDATE()`, `NOW()`	`CURRENT_DATE`, `NOW()`
Truncate to month	`DATE_FORMAT(d, '%Y-%m-01')`	`DATE_TRUNC('month', d)`
Extract part	`MONTH(d)`, `YEAR(d)`, `DAYOFWEEK(d)`, `HOUR(ts)`	`EXTRACT(MONTH FROM d)`, `EXTRACT(DOW FROM d)`
Date difference in days	`DATEDIFF(d2, d1)`	`d2 - d1` (date minus date gives days)
Add interval	`DATE_ADD(d, INTERVAL 7 DAY)`	`d + INTERVAL '7 days'`
Format as text	`DATE_FORMAT(d, '%b %Y')`	`TO_CHAR(d, 'Mon YYYY')`
Timestamp to date	`DATE(ts)`	`ts::date`
Minutes between timestamps	`TIMESTAMPDIFF(MINUTE, t1, t2)`	`EXTRACT(EPOCH FROM (t2 - t1)) / 60`
Last 30 days	`d >= CURDATE() - INTERVAL 30 DAY`	`d >= CURRENT_DATE - INTERVAL '30 days'`

Date traps

- A timestamp `<= '2026-10-31'` misses everything after midnight on the 31st. Use `ts >= '2026-10-01' AND ts < '2026-11-01'`.
- Group by the truncated date, not the raw timestamp, or every second becomes its own group.
- If the table stores UTC but the business day is IST, convert before bucketing by day or hour.

Strings

Task	Syntax
Concatenate	`CONCAT(a, ' ', b)` or `a || b` (Postgres)

Task	Syntax
Case	`UPPER(s)`, `LOWER(s)`
Length	`LENGTH(s)` (`CHAR_LENGTH` for characters)
Substring	`SUBSTRING(s, start, len)`
Trim	`TRIM(s)`
Replace	`REPLACE(s, 'old', 'new')`
Position	`POSITION('x' IN s)`
Pattern match	`s LIKE '%biryani%'` (case-insensitive: `ILIKE` in Postgres, default in MySQL)
Regex	`s REGEXP '^[0-9]+$'` (MySQL), `s ~ '^[0-9]+$'` (Postgres)

8. Product analytics patterns

These are the question shapes a product analyst test is built from. Examples use the tables defined in section 10 and MySQL date functions; swap in the Postgres versions from section 7 as needed.

1. Active users (DAU, MAU, stickiness)

```
WITH daily AS (
    SELECT DATE(event_ts) AS d, COUNT(DISTINCT user_id) AS dau
    FROM events GROUP BY DATE(event_ts)
)
SELECT DATE_FORMAT(d, '%Y-%m-01') AS month,
       AVG(dau) AS avg_dau
FROM daily GROUP BY 1;
-- MAU = COUNT(DISTINCT user_id) grouped by month; stickiness = avg DAU / MAU
```

2. Funnel conversion (open, search, menu view, cart, order)

```
WITH steps AS (
    SELECT COUNT(DISTINCT CASE WHEN event_name = 'app_open'     THEN user_id
END) AS opened,
           COUNT(DISTINCT CASE WHEN event_name = 'menu_view'    THEN user_id
END) AS viewed_menu,
           COUNT(DISTINCT CASE WHEN event_name = 'add_to_cart'  THEN user_id
END) AS carted,
           COUNT(DISTINCT CASE WHEN event_name = 'order_placed' THEN user_id
END) AS ordered
    FROM events
    WHERE event_ts >= '2026-10-01' AND event_ts < '2026-11-01'
)
SELECT *, ROUND(100.0 * carted / viewed_menu, 1) AS menu_to_cart_pct,
          ROUND(100.0 * ordered / carted, 1)     AS cart_to_order_pct
FROM steps;
```

This counts users who did each step at any time. For a strictly ordered funnel, self-join on `user_id` with `later.event_ts > earlier.event_ts`, or compare `MIN(event_ts)` per step per user.

3. Day-N retention by signup cohort

```
WITH first_order AS (
    SELECT user_id, MIN(order_date) AS first_date
    FROM orders WHERE status = 'delivered' GROUP BY user_id
)
SELECT f.first_date AS cohort,
       COUNT(DISTINCT f.user_id) AS cohort_size,
       COUNT(DISTINCT CASE WHEN DATEDIFF(o.order_date, f.first_date) = 7 THEN
o.user_id END) AS d7_users,
       ROUND(100.0 * COUNT(DISTINCT CASE WHEN DATEDIFF(o.order_date,
f.first_date) = 7
                                         THEN o.user_id END) / COUNT(DISTINCT
f.user_id), 1) AS d7_pct
FROM first_order f
LEFT JOIN orders o ON o.user_id = f.user_id AND o.status = 'delivered'
GROUP BY f.first_date;
```

For monthly retention, bucket both dates to the month and use month difference. Decide whether "retained" means exactly day 7 or any time within 7 days, and say which you chose.

4. New vs repeat customers, repeat rate

```
WITH u AS (
    SELECT user_id, COUNT(*) AS n FROM orders WHERE status = 'delivered'
GROUP BY user_id
)
SELECT COUNT(*) AS buyers,
       SUM(n >= 2) AS repeat_buyers,                          -- Postgres:
COUNT(*) FILTER (WHERE n >= 2)
       ROUND(100.0 * SUM(n >= 2) / COUNT(*), 1) AS repeat_rate_pct
FROM u;
```

To label each order as new or repeat, use `ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY order_ts) = 1` for new.

5. Month-over-month growth

```
WITH m AS (
    SELECT DATE_FORMAT(order_date, '%Y-%m-01') AS month, SUM(order_value) AS
gmv
    FROM orders WHERE status = 'delivered' GROUP BY 1
)
SELECT month, gmv,
       ROUND(100.0 * (gmv - LAG(gmv) OVER (ORDER BY month))
             / NULLIF(LAG(gmv) OVER (ORDER BY month), 0), 2) AS mom_pct
FROM m;
```

6. Consecutive-day streaks (gaps and islands: subtract a row number from the date, and consecutive dates share the same result)

```
WITH d AS (SELECT DISTINCT user_id, order_date FROM orders),
g AS (
    SELECT user_id, order_date,
           DATE_SUB(order_date, INTERVAL ROW_NUMBER() OVER (PARTITION BY
user_id ORDER BY order_date) DAY) AS grp
    FROM d
)
SELECT user_id, MIN(order_date) AS streak_start, MAX(order_date) AS
streak_end, COUNT(*) AS days
FROM g GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
```

7. Median (no `MEDIAN()` in MySQL)

```
-- PostgreSQL
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY delivery_mins) FROM
orders;

-- MySQL 8
SELECT AVG(delivery_mins) FROM (
    SELECT delivery_mins,
           ROW_NUMBER() OVER (ORDER BY delivery_mins) AS rn,
           COUNT(*) OVER () AS cnt
    FROM orders
) t WHERE rn IN (FLOOR((cnt + 1) / 2), CEIL((cnt + 1) / 2));
```

8. Share of total and the 80/20 cut

```
WITH r AS (SELECT restaurant_id, SUM(order_value) AS rev FROM orders GROUP BY
restaurant_id)
SELECT restaurant_id, rev,
       SUM(rev) OVER (ORDER BY rev DESC) / SUM(rev) OVER () AS cum_share
FROM r;
```

9. Service-level metrics

```
SELECT city,
       ROUND(AVG(delivery_mins), 1) AS avg_mins,
       ROUND(100.0 * AVG(delivery_mins <= 30), 1) AS pct_within_30,     --
Postgres: AVG((delivery_mins <= 30)::int)
       ROUND(100.0 * AVG(status = 'cancelled'), 1) AS cancel_pct
FROM orders GROUP BY city;
```

The average of a 0/1 flag is the percentage that meets the condition, which is the shortest way to write most rate metrics.

Metric definitions to state out loud in the test: GMV = sum of order value (decide on cancelled orders and fees); AOV = GMV / delivered orders; order frequency = orders / active buyers in the period; retention = share of a cohort active again after N days; conversion = users at step k+1 / users at step k.

9. Common mistakes and edge cases

Mistake	Fix
`COUNT(*)` where unique users were asked for	`COUNT(DISTINCT user_id)`
Joining before aggregating, which inflates sums (fan-out)	Aggregate each table to the join grain in a CTE first
Filtering a LEFT JOIN's right table in WHERE	Put the condition in `ON`
`= NULL` or `<> NULL`	`IS NULL`, `IS NOT NULL`
`NOT IN` with a subquery that can return NULL	`NOT EXISTS` or LEFT JOIN anti-join
Integer division giving 0	Multiply by `1.0` first
Dividing by a zero denominator	`NULLIF(denominator, 0)`
`BETWEEN` on timestamps, missing the last day	Half-open range: `>= start AND < next_start`
Selecting a column not in `GROUP BY`	Add it, or aggregate it
Window function in WHERE	Wrap in a CTE or subquery, filter outside
`RANK()` where "top N distinct" was asked	`DENSE_RANK()`; use `ROW_NUMBER()` only when exactly N rows are wanted
Counting cancelled or test orders in a business metric	Filter `status`, and state the assumption
`UNION` when duplicates should stay	`UNION ALL` (also faster)
`ORDER BY` inside a CTE or subquery expecting it to stick	Order only in the final SELECT
`LAST_VALUE` returning the current row	Add `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`

Edge cases to mention before you are asked

- Users with zero orders: use a LEFT JOIN so they show as 0 rather than disappearing.

- Days with no data: join to a calendar table or a generated date series, or the row is silently missing.
- Ties: say how you handled them (RANK vs DENSE_RANK vs ROW_NUMBER).
- Duplicate event rows: de-duplicate with `DISTINCT` or `ROW_NUMBER() = 1` before counting.
- Small denominators: a 100% rate on 2 orders is noise, so add a minimum-volume `HAVING`.

Speed (when the test notes performance)

- Filter early and select only needed columns. Avoid `SELECT *` in the final query.
- Avoid functions on indexed columns in WHERE (`WHERE DATE(order_ts) = ...` blocks index use; use a range).
- `EXISTS` is usually at least as fast as `IN` for large subqueries. `UNION ALL` beats `UNION`.
- `EXPLAIN` shows the plan; mention indexes on join and filter keys if asked how to speed a query up.

10. Practice schema and questions

This is an invented schema in the style of a food-delivery product, used by every example above. Try each question before reading the answer.

Table	Columns
`users`	user_id, signup_date, city, is_gold
`restaurants`	restaurant_id, name, city, cuisine, rating
`orders`	order_id, user_id, restaurant_id, order_ts, order_date, status (delivered / cancelled), order_value, delivery_fee, delivery_mins
`order_items`	order_id, item_id, qty, price
`events`	user_id, event_ts, event_name (app_open, search, menu_view, add_to_cart, order_placed), session_id

Q1. Cities with more than 1000 delivered orders in October 2026, with their AOV.

```
SELECT city, COUNT(*) AS orders, ROUND(AVG(order_value), 2) AS aov
FROM orders
WHERE status = 'delivered' AND order_date >= '2026-10-01' AND order_date <
'2026-11-01'
GROUP BY city HAVING COUNT(*) > 1000 ORDER BY orders DESC;
```

Q2. Users who signed up but never placed an order.

```
SELECT u.user_id FROM users u
LEFT JOIN orders o ON o.user_id = u.user_id
WHERE o.order_id IS NULL;
```

Q3. Top 3 restaurants by delivered revenue in each city.

```
WITH rev AS (
  SELECT r.city, r.restaurant_id, SUM(o.order_value) AS revenue
  FROM orders o JOIN restaurants r ON r.restaurant_id = o.restaurant_id
  WHERE o.status = 'delivered' GROUP BY r.city, r.restaurant_id
)
SELECT * FROM (
  SELECT *, DENSE_RANK() OVER (PARTITION BY city ORDER BY revenue DESC) AS rk
FROM rev
) t WHERE rk <= 3;
```

Q4. Each user's second order date.

```
SELECT user_id, order_date FROM (
  SELECT user_id, order_date,
         ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY order_ts) AS rn
  FROM orders WHERE status = 'delivered'
) t WHERE rn = 2;
```

Q5. Cancellation rate by cuisine, only for cuisines with at least 50 orders.

```
SELECT r.cuisine, COUNT(*) AS orders,
       ROUND(100.0 * AVG(o.status = 'cancelled'), 2) AS cancel_pct
FROM orders o JOIN restaurants r ON r.restaurant_id = o.restaurant_id
GROUP BY r.cuisine HAVING COUNT(*) >= 50 ORDER BY cancel_pct DESC;
```

Q6. Orders where the value is above that user's own average order value.

```
SELECT order_id, user_id, order_value FROM (
  SELECT *, AVG(order_value) OVER (PARTITION BY user_id) AS user_avg FROM
orders
) t WHERE order_value > user_avg;
```

Q7. Orders by hour of day, and the peak hour.

```
SELECT HOUR(order_ts) AS hr, COUNT(*) AS orders   -- Postgres: EXTRACT(HOUR
FROM order_ts)
FROM orders GROUP BY HOUR(order_ts) ORDER BY orders DESC;   -- first row is
the peak
```

Q8. Share of new buyers who reorder within 30 days of their first order.

```
WITH f AS (
  SELECT user_id, MIN(order_date) AS first_date FROM orders WHERE status =
'delivered' GROUP BY user_id
)
SELECT ROUND(100.0 * COUNT(DISTINCT CASE WHEN o.order_date > f.first_date
                                          AND DATEDIFF(o.order_date,
f.first_date) <= 30
                                         THEN f.user_id END) / COUNT(DISTINCT
f.user_id), 1) AS reorder_30d_pct
FROM f JOIN orders o ON o.user_id = f.user_id AND o.status = 'delivered';
```

Q9. Gold vs non-gold: users, orders per user and AOV.

```
SELECT u.is_gold, COUNT(DISTINCT u.user_id) AS buyers,
       ROUND(COUNT(*) * 1.0 / COUNT(DISTINCT u.user_id), 2) AS
orders_per_buyer,
       ROUND(AVG(o.order_value), 2) AS aov
FROM users u JOIN orders o ON o.user_id = u.user_id AND o.status =
'delivered'
GROUP BY u.is_gold;
```

Q10. Restaurants rated below their city's average rating.

```
SELECT restaurant_id, name, city, rating FROM (
  SELECT *, AVG(rating) OVER (PARTITION BY city) AS city_avg FROM restaurants
) t WHERE rating < city_avg;
```

Q11. Duplicate order rows (same order_id more than once).

```
SELECT order_id, COUNT(*) FROM orders GROUP BY order_id HAVING COUNT(*) > 1;
```

Q12. Month-over-month change in daily active users. Combine the DAU CTE from pattern 1 with `LAG` over the monthly average from pattern 5.

Final check before you submit: does each query return the grain the question asked for, are cancelled orders handled on purpose, and have you run it on a few rows by hand?